PyPIGuard: A Lightweight, Explainable Machine Learning Tool for Pre-Install Detection of Malicious PyPI Packages

Rahat Ahmed

Department of Computer Science and Engineering, BRAC University, Dhaka, Bangladesh

Abstract

Open-source package registries such as PyPI are increasingly targeted by attackers, yet a benchmark study of deployed detection tools reports false-positive rates between 15% and 97%, well above the near-zero tolerance PyPI maintainers require for automated adoption [6]. PyPIGuard is an open-source, deployable tool that classifies PyPI packages as malicious or benign using only static code, Abstract Syntax Tree (AST), and bytecode-level structural features – without relying on a large language model (LLM) or executing the package. It is deployed as a live web application with a real-time scanner, a file-upload scanner, and a Jupyter notebook malware-import scanner. Trained on 9,723 real packages – the entire available malicious archive plus freshly-sampled benign packages – it achieves 0.9993 ROC-AUC and a 0.95 precision / 0.99 recall on the malicious class on independent held-out data, with a median model-inference time of 0.18 ms per package, remaining robust under adversarial evasion and temporal-generalization testing.

Keywords: malware detection, software supply chain security, machine learning, static code analysis, PyPI, explainable AI

Required Metadata

Current code version

Main text

1. Motivation and significance
   Open-source package registries such as PyPI are foundational to modern software development, but they are increasingly targeted by attackers who publish malicious packages – often through typosquatting

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/rahat239/PyPIguard-Research`
C3	Legal Code License	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python 3.12, scikit-learn, Flask, HTML/CSS/JavaScript
C6	Compilation requirements, operating environments & dependencies	Python ≥3.10; `pip install -r requirements.txt` (pandas, numpy, scikit-learn, joblib, requests, flask, shap); no OS-specific dependencies
C7	If available Link to developer documentation/manual	`README.md` in the code repository (C2)
C8	Support email for questions	rahatahmed537@gmail.com

Table 1: Code metadata (mandatory)

popular libraries such as `beautifulsoup4`, `matplotlib`, and `requests`. In the real `event-stream` incident (2018) [2], an attacker was granted maintainer access to a popular package simply by offering to help maintain it, and the compromised package reached over 1,600 downstream projects before discovery.

Existing approaches sit at two extremes, leaving a gap PyPIGuard targets. Lightweight metadata-only classifiers are fast but easily gamed once attackers adapt. Heavyweight behavioral models – e.g., Cerebro [3], requiring 14–25 hours of training and reporting an 81.5% false-positive rate on PyPI in its own evaluation – are too slow or imprecise for real-time, pre-install scanning. Vu et al.'s benchmark [6] found 15–97% false-positive rates across deployed PyPI scanners, and reports administrators consider rates above ∼0.01% operationally unacceptable given daily release volume.

The gap PyPIGuard addresses: a static-analysis classifier simultaneously fast enough for real-time use, explainable per-prediction, and validated against adversarial evasion, temporal drift, and cross-ecosystem transfer – none jointly reported for the tools in [6]. It is built on the dataset released with [1] and PyPI's official index.

In practical terms: a developer supplies a package name, archive, or notebook; PyPIGuard analyzes it without executing it and returns a

CLEARED/FLAGGED verdict, a confidence score, and the specific patterns that drove the decision.

2. Software description

2.1 **Software architecture.** Figure 1 shows the end-to-end pipeline. A submitted package is first checked against a **verified-lookup table** of 5,900 packages with ground-truth labels; if found, an instant dataset-backed verdict is returned, visually marked "verified" rather than model-predicted. Otherwise, three sequential stages run: (1) a **feature-extraction layer** retrieves the source (or accepts a local upload) and computes 35 static features via regex, Python's `ast` module, and `compile()`-based bytecode inspection, none requiring execution; (2) a **classification layer** – a single Random Forest classifier – outputs a prediction and confidence score; (3) a **delivery layer** (Flask) formats the verdict with a SHAP [8] explanation and flagged patterns. An **out-of-distribution safety gate** between (2) and (3) flags verdicts as unreliable when a package's size falls far outside the training range, a behavior added after deployment testing (Section 4).

2.2 **Software functionalities.** Three input modes share one verdict format. *Package-name scanning* downloads and analyzes a package live via PyPI's API. *Archive-file scanning* accepts a local `.tar.gz`/`.zip`/`.whl` directly – useful for historical malware PyPI has since removed. Extraction is hardened against path-traversal ("zip-slip") attacks, validated against a crafted adversarial archive. *Notebook scanning* parses every `.ipynb` code cell, extracts `import`/`pip install` references, excludes Python's ∼300 standard-library modules, resolves import-to-distribution mismatches (e.g., `cv2`→`opencv-python`), and reports how many referenced packages are known or suspected malware before execution. Every verdict includes a CLEARED/FLAGGED label, confidence, flagged patterns (e.g., `eval()` usage, `setup.py cmdclass` hooks, high-entropy blobs), and a bytecode-level detector for calls like `getattr(__builtins__,...)` that identifies dynamic dangerous-function invocation independent of the string-construction technique used to hide it. Current limitations: no dynamic/execution-based analysis, and reduced reliability for packages structurally far larger than the training data (flagged via the safety gate above).

2.3 **Sample code snippets.** A representative scan:

```
import ast_features, regex_features, joblib
feats = {**regex_features.extract(source_dir),
         **ast_features.extract_ast_features(source_dir)}
```

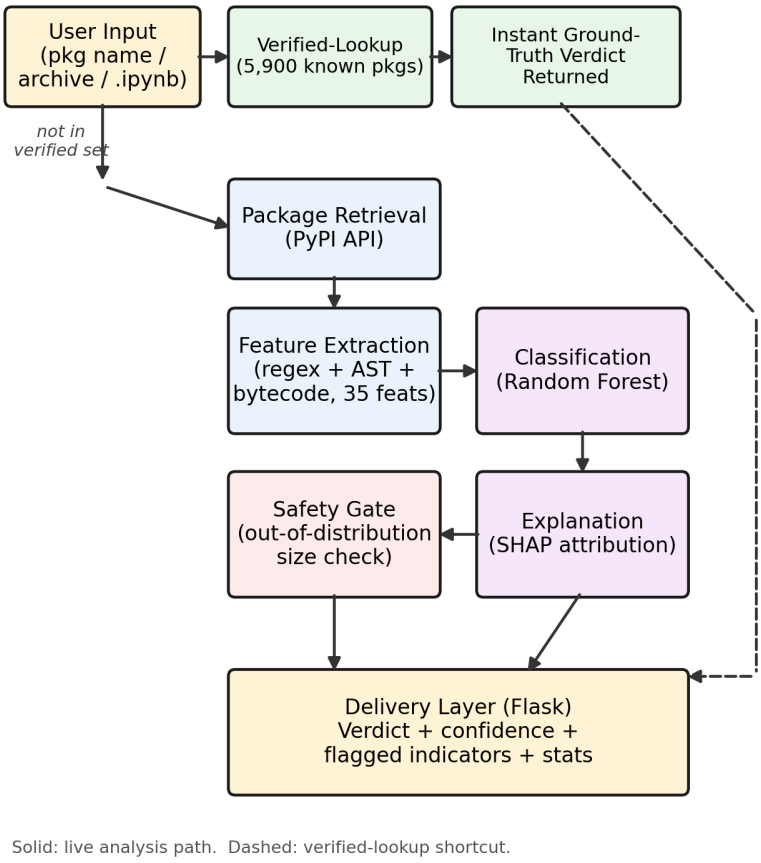


Figure 1: PyPIGuard architecture. Solid arrows: live analysis path. Dashed arrow: verified-lookup shortcut.

```
model = joblib.load("final_integrated_model.joblib")
proba = model.predict_proba([feats])[0]
verdict = "malicious" if proba[1] > 0.5 else "benign"
explanation = shap_explainer(feats)
```

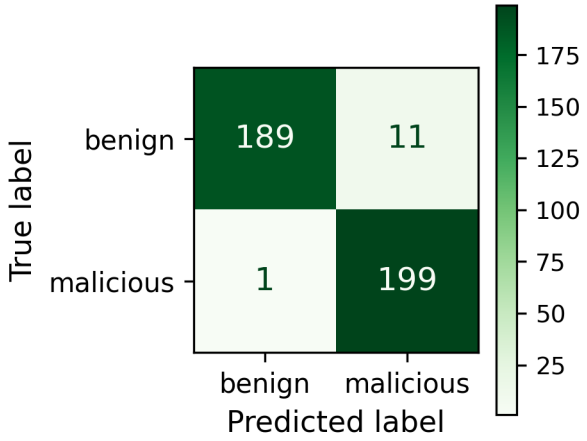
The web application wraps this pipeline behind the three input modes; full implementation is in the code repository (C2).

3. Illustrative examples
   A user submits a package name (e.g., `requests`) or uploads a local archive through the web interface. Figure 2(a) shows the resulting interface for `requests`: a stamped CLEARED verdict at 54.7% confidence and the flagged code patterns contributing to it – the same package and confidence-calibration case discussed in Section 4. Figure 2(b) shows the notebook-upload interface alongside a verified-lookup result: a known-malicious package (`selfcraftint`) instantly flagged at 100% confidence from the dataset-backed lookup table described in Section 2, together with the interface's stated limitations.
4. Impact
   Evaluation methodology. The final model was trained on 9,723 labeled packages (7,960 malicious, 1,763 benign; "at scale" refers to this set, covering the entire available archive from [1], a $\sim$3.9$\times$ increase over an earlier 2,500-sample version) and validated using 5-fold **GroupKFold** cross-validation, with an independent 400-package held-out set reported here (zero package-name overlap, verified programmatically). Groups were defined by typosquat family rather than by individual package: malicious samples that are near-duplicate variants of the same base package (e.g., dozens of typosquats of `requests`) are grouped together so near-duplicates never split across train and test, which a naive random or single-package-based split would not prevent. Results: 0.9993 ROC-AUC, 0.9993 PR-AUC, 0.95 precision, 0.99 recall on the malicious class (Figure 3a).
   Robustness. *Temporal*: training only on packages dated before a fixed cutoff and testing exclusively on later packages yields 0.984 ROC-AUC, confirming generalization to malware discovered after training. *Adversarial evasion*: 100 detected malicious packages were paired with behavior-preserving disguises – e.g., rewriting a direct `eval(x)` call as an equivalent `getattr`-based indirection that evades a literal string match. The **susceptibility gap** – the fraction of these that evade re-detection – fell from 9% (regex-only) to 2% (AST + bytecode features), an improvement but not a complete fix, reported as a limitation. *Cross-ecosystem*: applying the PyPI-trained model unretrained to real npm packages gives 0.818 ROC-AUC (vs. 0.999 in-ecosystem), confirmed

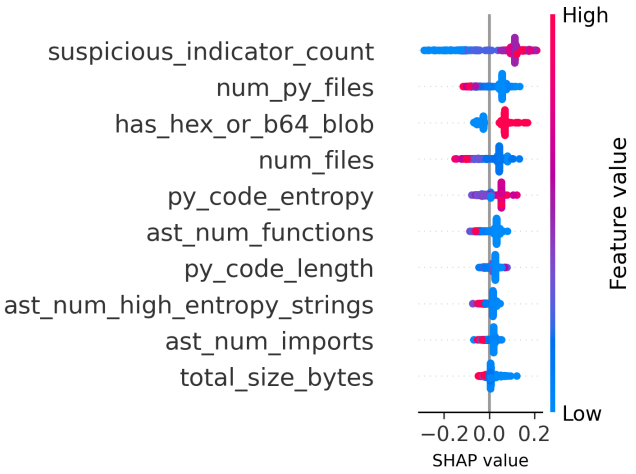
(a) Package scan result (`requests`)

(b) Notebook upload & verified-lookup result

Figure 2: PyPIGuard web interface: (a) live scan of `requests`, showing the CLEARED verdict, confidence score, and flagged patterns discussed in Section 4; (b) notebook-scanning upload widget and a verified-lookup verdict for a known-malicious training-set package.



(a) Confusion matrix



(b) SHAP summary

Figure 3: (a) Confusion matrix, held-out set: 189/200 benign correctly cleared (11 FP), 199/200 malicious correctly flagged (1 FN). (b) SHAP global feature importance (top 10 features): horizontal position shows impact on model output, color shows feature value.

consistent at both 2,500- and 9,723-sample scales (0.815 vs. 0.818, hence "across two dataset scales"); recall drops to 0.54, so PyPIGuard should not be deployed on npm without retraining.

For context, Cerebro [3] reports an 81.5% false-positive rate on PyPI in its own evaluation; PyPIGuard's held-out false-positive rate is 5.5%. MalGuard [4] reports 91.0% precision and 94.0% recall in its own evaluation (independently reproduced by a third-party comparison study); PyPIGuard's held-out precision and recall (95%, 99%) exceed both figures. Neither comparison is head-to-head on identical data – both baselines use their own evaluation datasets – but together they indicate PyPIGuard operates in a substantially different, more conservative operating regime than either representative baseline.

Explainability. Figure 3b shows SHAP global feature importance for the top 10 features. `suspicious_indicator_count` (the aggregate count of triggered regex-based patterns) is the single most influential feature, followed by `num_py_files`, `has_hex_or_b64_blob`, and `num_files`: packages triggering more suspicious patterns simultaneously, or containing fewer Python files overall, are pushed toward malicious, consistent with thin, single-purpose typosquats; larger file counts push toward benign, consistent with legitimate packages carrying more supporting files (tests, documentation, packaging metadata) than minimal payloads. This is also the feature-interaction mechanism underlying the version-stability finding above: vendored third-party code inflates `suspicious_indicator_count` independent of a package's own behavior.

Latency (deployment-testing finding). We distinguish three previously conflated measures: *model inference* (0.18 ms median), *feature extraction* (0.2–1 ms), and *end-to-end scanning* (PyPI network round-trip plus hosting overhead). A reviewer-observed ∼20s scan of `requests` is attributable entirely to download time and free-tier hosting cold-start, not model inference, which stays sub-millisecond; this is now stated as an explicit deployment limitation.

Confidence calibration (deployment-testing finding). `requests` was classified benign at 54.3% confidence during reviewer testing (54.7% on our own re-verification of the same version, confirming stability across repeated scans) – correct, but near the decision boundary. Random Forest probability estimates are not calibrated probabilities: bagging averages predictions across trees with high individual variance, which is established to bias ensemble predictions away from the extremes (near 0 or 1) toward the middle of the range [7], independent of any single input's characteristics. We additionally tested version stabil-

ity directly: scanning five real `requests` releases spanning 2011–2026 (v0.2.0, v0.9.2, v2.2.0, v2.17.0, v2.34.2) found 3 of 5 **misclassified as malicious**, one (v2.17.0) at 93.7% confidence. Investigating the cause, we found these releases bundle vendored copies of their own dependencies (e.g., urllib3, chardet) directly in the source tree, an older packaging convention; this vendored third-party code triggers multiple suspicious-pattern features cumulatively (including a genuine match on our `getattr` obfuscation detector in v2.2.0's vendored code), inflating the aggregate suspicious-indicator count regardless of `requests`' own code. This reveals a genuine construct-validity limitation, not merely a confidence-calibration nuance: PyPIGuard's current static features cannot distinguish a package's own suspicious behavior from suspicious patterns present in legitimately vendored third-party code, and this is reported here as an identified limitation rather than a solved problem. The current version (v2.34.2, no longer vendoring dependencies) and one older version (v0.9.2) are both classified correctly; the size-based explanation offered for the single-version case above (85 files/390,307 characters vs. a training median of 16/19,431) remains a contributing but not sole factor. A path-traversal ("zip-slip") vulnerability, found and patched via a crafted adversarial test archive prior to deployment, is documented here for the same transparency reason.

Practical impact. Intended users are developers, students, and CI/CD pre-install gates deciding whether to install a dependency. A false negative risks installing malware; a false positive costs a manual review. Given the 5.5% FP / 0.5% FN rates and explicit uncertainty flagging, PyPIGuard is a decision-support triage tool, not an autonomous blocker. It is deployed as a public, live tool (Section 5), testable against real historical malware no longer on the live registry, since PyPI removes malicious packages once discovered.

Future work includes extending static learning to other ecosystems beyond the partial PyPI-to-npm transfer above, and combining PyPIGuard's tabular features with sequence-order information – a separate matched comparison found a simplified Cerebro-inspired [3] sequence proxy modestly outperforming PyPIGuard (0.983 vs. 0.967 ROC-AUC), suggesting sequence information carries signal the flat feature vector misses.

5. Conclusions

PyPIGuard is an open-source, deployed, explainable tool for pre-install detection of malicious PyPI packages, validated through temporal, cross-ecosystem, and adversarial-evasion testing not typically reported together for tools of this scope. It shows that a static, LLM-free analysis

approach operates in a substantially different false-positive regime than a benchmarked deep-learning baseline [3], while transparently reporting its boundaries: cross-ecosystem transfer is only partial, adversarial evasion is reduced but not eliminated, static analysis alone leaves dynamic/behavioral attack surfaces uncovered pending access to properly isolated execution infrastructure, and the current free-tier deployment does not represent production-grade latency despite sub-millisecond model inference.

Acknowledgements

The author thanks the maintainers of the `pypi_malregistry` dataset [1] and the Datadog Security Labs malicious-software-packages dataset [5], both of which made this work possible.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used ChatGPT (OpenAI) to assist with drafting and restructuring portions of the manuscript, correcting grammatical errors, and refining writing style. After using this tool, the author reviewed, edited, and verified all content for technical accuracy and takes full responsibility for the content of the published article.

References

[1] W. Guo, Z. Xu, C. Liu, C. Huang, Y. Fang, Y. Liu, An Empirical Study of Malicious Code In PyPI Ecosystem, in: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2023, pp. 166–177.

[2] M. Ohm, H. Plate, A. Sykosch, M. Meier, Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks, DIMVA, 2020.

[3] J. Zhang, K. Huang, B. Chen, C. Wang, Z. Tian, X. Peng, Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI Using a Single Model of Malicious Behavior Sequence, ACM Transactions on Software Engineering and Methodology, 2025.

[4] X. Gao et al., MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem, USENIX Security, 2025.

[5] Malicious Software Packages Dataset, Datadog Security Labs, 2023.

[6] D.-L. Vu, Z. Newman, J.S. Meyers, A Benchmark Comparison of Python Malware Detection Approaches, arXiv:2209.13288, 2022.

[7] A. Niculescu-Mizil, R. Caruana, Predicting Good Probabilities With Supervised Learning, in: Proceedings of the 22nd International Conference on Machine Learning (ICML), 2005, pp. 625–632.

[8] S.M. Lundberg, S.-I. Lee, A Unified Approach to Interpreting Model Predictions, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 4765–4774.

Current executable software version

Live demonstration available at: `https://rahat239.github.io/Malware-classifier/`